常用模块

- [ ] 必须掌握time模块
- [ ] 必须掌握random模块
- [ ] 必须掌握os模块、了解sys模块
- [ ] 必须掌握json模块

一、time模块

Python的`time`模块提供了各种与时间相关的函数，这些函数对于需要**处理时间、日期**时非常有用。

时间戳（timestamp）：具体时间距离历元(1970年01月01日 00:00:00:000) 经过的毫秒数，用float类型存储。时间戳可以很容易被解释(转换)为年、月、日、小时、分钟和秒值。

- time.time( ): 获得当前时间的时间戳，返回的是float类型。
- time.sleep(second): 休眠多少秒。

```python
import time

# 1 time() 获取当前时间戳
ts = time.time()
print(ts)

# 2 sleep() 程序休眠
# time.sleep(3)
```

1、获取当前时间

- time.localtime(timestamp)：获得当前时间的结构化时间，参数默认当前时间的时间戳，可以自己给

```python
import time

# 1 time() 获取当前时间戳
ts = time.time()
print(ts)

# 2 sleep() 程序休眠
# time.sleep(3)

# 3 localtime() 返回结构化时间
st = time.localtime()
print(f"{st[0]}年{st[1]}月{st[2]}日")
print(f"星期{st[6]+1}")
```

结构化时间(struct_time)：struct_time元组共有9个元素(**年，月，日，时，分，秒**，一年中第几周，一年中第几天，夏令时)

索引（Index）	属性（Attribute）	值（Values）
0	tm_year（年）	比如2011
1	tm_mon（月）	1 - 12
2	tm_mday（日）	1 - 31
3	tm_hour（时）	0 - 23
4	tm_min（分）	0 - 59
5	tm_sec（秒）	0 - 60
6	tm_wday（weekday）	0 - 6（0表示周一）
7	tm_yday（一年中的第几天）	1 - 366
8	tm_isdst（是否是夏令时）	认为0

2、格式化和解析操作

- time.strftime(format,struct_time): 根据指定格式将时间元组转换为字符串，参考格式化时间字符串化对应表
- time.strptime(string_time,format): 将时间字符串解析为时间元组struct_time

格式化时间字符串化对应表

```
%Y  Year with century as a decimal number.
%m  Month as a decomal number [01,12].
%d  Day of the month as a decomal number [01, 31].
%H  Hour (24-hour clock) as a decimal number [00,23].
%M  Minute as a decimal number [00,59].
%S  Second as a decimal number [00,61].
%z  Time zone offset from UTC.
%a  Locale's abbreviated weekday name.  # 星期的缩写
%A  Locale's full weekday name.         # 星期的全写
%b  Locale's abbreviated month name.    # 月份的缩写
%B  Locale's full month name.           # 月份的全写
%X  Locale's appropriate time representation.  # 冒号分割的时间
```

案例：日期时间和字符串的转换

```
# 4 需求1: 把当前日期时间格式化成字符串(2025-01-25 22:02:55)
str = time.strftime('%Y-%m-%d %X', time.localtime())
print(str)

# 5 需求2: 把字符串'2025-01-25 22:02:55'转换成日期时间类型
tm = time.strptime('2025-01-25 22:02:55', '%Y-%m-%d %X')
print(tm)
```

二、random模块

随机数模块，用于产生随机数

1、random的常用方法

- random.random(): 生成一个[0.0, 1.0)范围内的随机小数。
- random.randint(a,b): 生成一个在[a, b]范围内的随机整数（包括a和b）
- random.uniform(a,b): 返回一个在 [a, b] 区间内的随机小数（包括 a 和 b）
- randrange(start, stop[, step])：从指定范围内按照指定基数递增的集合中获取一个随机数，类似于`range()`函数。
- random.choice(有序的iterable): 从有序的可迭代对象中随机选择一个元素
- random.shuffle(iterable): 将有序可迭代对象中的元素打乱顺序。（类似洗牌）

```python
import random

# 1 random()
r1 = random.random()
print(r1)

# 2 randint()
r2 = random.randint(1,10)
print(r2)

# 3 uniform
r3 = random.uniform(1.0,3.0)
print(r3)

# 4 randrange(start, end, step)
# 使用randrange生成一个在[0, 10]之间的偶数
r4 = random.randrange(0, 10, 2)
print(r4)

r5 = random.choice(range(1, 10))
print(r5)

list = [1, 2, 3, 4, 5]
random.shuffle(list)
print(list)
```

三、os模块

os 模块提供了很多允许你的程序与操作系统直接交互的功能

- **文件和目录操作**：创建、删除、重命名文件或目录；获取当前工作目录等。
- **路径操作**：处理不同操作系统下的路径格式（如拼接路径）。

案例1：os模块对文件或目录的操作

```python
import os

# 执行 shell 命令
# os.system('start cmd')  # 打开cmd
# os.system('start calc') # 打开calc

# 获取当前文件所在的路径目录
print(os.getcwd())
```

```
print(__file__)

# 创建目录
# os.mkdir('test')  # 在当前目录下创建目录，  如果存在创建失败
# os.rmdir('test')  # 删除目录必须为空目录

# 对文件重命名
os.rename('test\\a.py','test\\b.py')
```

案例2：os.path 模块

```
import os

# 1 判断文件是否存在
print(os.path.exists(r'D:\2025-py\python-workspace\day10-常用模块\test\b.py'))
# 2 获取当前文件路径
print(__file__)

# 3 获取当前文件所在目录
print(os.path.dirname(__file__))
# 4 获取文件名
print(os.path.basename(__file__))

# 4 join(path, name) 拼接目录和文件名
# 需求: 构建一个指定文件的路径
cur_path = os.path.dirname(__file__)
file_name = os.path.join(cur_path, 'c.py')
print(file_name)
```

四、sys模块（了解）

sys 模块主要负责与 Python 解释器进行交互，该模块提供了一系列用于控制 Python 运行时环境的不同部分（函数和变量等）。

- **命令行参数**：访问通过命令行传递给Python脚本的参数。
- **退出程序**：提前终止程序并可选地返回一个状态码。
- **Python路径管理**：查看和修改Python解释器搜索模块的路径列表。
- **标准输入输出**：直接与标准输入输出流交互。

1、获取参数列表

argv：返回传递给 Python 脚本的命令行参数列表。

```
import sys

print("Script name:", sys.argv[0])
if len(sys.argv) > 1:
    print("Arguments passed:", sys.argv[1:])
else:
    print("No arguments passed.")
```

要测试此代码，请从命令行运行脚本并附带参数，例如 `python script.py arg1 arg2`。

2、退出当前程序

sys.exit(arg)——程序中间的退出，

- arg = 0：表示正常退出；
- arg != 0：表示异常退出，会引发一个SystemExit异常，从而可以在主程序中捕获该异常。

```
import sys

print('Hello--')
# sys.exit()
try:
    sys.exit(3)
except SystemExit:
    print("SystemExit异常")

print('end---')
```

3、标准输入输出

```
import sys

# 1. 标准输出stdout
# 下面两行代码等价
sys.stdout.write('标准输出stdout' + '\n')
print('标准输出print')

# 2. 标准输入stdin
s1 = input(">>")
print(s1)
s2 = sys.stdin.readline()
print(s2)

# 3.错误输出stderr
sys.stderr.write('错误输出stderr')
```

4、返回模块的搜索路径列表

```
import sys
print(sys.path)
```

五、json模块

`json.dumps()`：将Python对象转换为JSON字符串。

`json.loads()`：解析JSON字符串并转换为Python对象。

`json.dump()`分别用于将Python对象写入文件作为JSON格式

`json.load()`：从文件中读取JSON数据到Python对象。

1、基本的JSON操作

将Python字典转换为JSON字符串

```python
import json

# Python字典
data = {
    "name": "张三",
    "age": 18,
    "phone": "18620015858"
}

# 转换为JSON字符串
json_str = json.dumps(data, ensure_ascii=False)  # 使用ensure_ascii=False来正确显示中文
print("JSON字符串:", json_str)
```

解析JSON字符串为Python字典

```python
# JSON字符串
json_str = '{"name": "张三", "age": 18, "phone": "18620015858"}'

# 解析为Python字典
data = json.loads(json_data)
print("Python字典:", data)
```

2、文件操作

将Python字典写入JSON文件

```python
data = {
    "name": "张三",
    "age": 18,
    "phone": "18620015858"
}

with open('data.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False)
print("数据已保存到data.json")
```

从JSON文件读取数据到Python字典

```python
with open('data.json', 'r', encoding='utf-8') as f:
    loaded_data = json.load(f)
print("从文件加载的数据:", loaded_data)
```

注意事项

- `ensure_ascii=False` 参数用于确保非ASCII字符（如中文）能够正确地显示在输出中。
- 当处理文件时，请记得指定正确的编码（例如UTF-8），以避免编码问题